

EGATE TRAINING PROGRAM

Advanced Electronics & Robotics Track: Final Project Documentation

Section 1: Project Overview

- **Project Title:** Autonomous GPS Waypoint Navigation Drone with Obstacle Avoidance
- **Trainee:** Bilisuma Jabesa Merga
- **Mentors:** Mr. Kinfegebriel Kebede and Mr. Solomon Ayalew
- **Date:** September 16, 2026

Section 2: Abstract & Engineering Objectives

This project presents the design, simulation and embedded implementation of an autonomous navigation system. Modern autonomous drones require reliable spatial orientation, collision evasion and failsafe return mechanisms when deployed in urban. The system uses Python ground station with an SRAM-optimized Arduino Uno (ATmega328P) flight controller and a comprehensive Proteus virtual simulation testbed.

The Python ground station queries real-world geospatial street and transit vector networks via OpenStreetMap (OSMNx and NetworkX), generates an interactive HTML flight trajectory and compiles limited sampled latitude/longitude coordinates into microcontroller headers. Onboard, the drone executes a non-blocking Finite State Machine (FSM) reading from a NEO-6M GPS receiver, angular kinematics from an MPU-6050 6-axis IMU, proximity alerts from an infrared (IR) obstacle sensor and orientation integrity from a ball tilt switch. Differential thrust motor control guides the quadcopter along sequential waypoints, initiates evasive maneuvers upon obstacle detection, holds at the target destination and autonomously executes a Return-to-Home (RTH) reverse landing sequence.

- **Core Objective 1:** Develop an automated GIS route-planning pipeline that downloads real-world geographic coordinates, calculates shortest-path trajectories, down samples waypoints to strictly adhere to 8-bit MCU SRAM constraints (<2 KB) and exports structured C++ flight headers.
- **Core Objective 2:** Architect a robust embedded flight controller firmware featuring an 8-state deterministic FSM, I2C IMU telemetry, real-time IR collision evasion and failsafe tilt-inversion motor shutdown.

Section 3: Hardware Architecture

The physical and simulated hardware architecture is engineered around the ATmega328P 8-bit RISC microcontroller operating at 16 MHz. The power distribution architecture consists of a dual-rail supply: a 9V external battery driving the four high-current DC coreless lift motors through individual 2N1711 NPN bipolar junction switching transistors, and a regulated 5V DC rail powering the MCU logic, sensors, and wireless transceivers. Sensor inputs include a U-Blox NEO-6M GPS module communicating via SoftwareSerial, an InvenSense MPU-6050 IMU connected to the hardware I2C bus, an analog/digital forward-facing IR obstacle sensor and an omnidirectional ball tilt switch for crash detection.

Item No.	Component Name	Specification / Model	Sub-system Role	Quantity
1	Microcontroller Unit (MCU)	Arduino Uno R3	Primary Flight Computer & FSM Controller	1
2	Inertial Measurement Unit	InvenSense MPU-6050 (3-Axis Gyro + 3-Axis Accel)	Pitch/Roll Attitude & Dynamic Angular Rate Sensing	1
3	Proximity / Collision Sensor	Active Infrared (IR) Proximity Sensor (400–1023 Thresh)	Forward Obstacle Detection & Evasion Trigger	1
4	Failsafe Tilt Switch	Omnidirectional Ball Tilt Switch (Rolling Ball Element)	Inversion / Airframe Crash Detection Failsafe	1
5	DC Actuators / Motors	Coreless High-Speed DC Motors / 5V DC Fans (FAN-DC)	Differential Aerodynamic Quadcopter Propulsion	4
6	Power Switching Transistors	2N1711 NPN BJT Transistors (TO-5 Metal Package)	High-Current Low-Side Motor Ground Switching	4
7	Base Current-Limiting Resistors	100Ω Metal Film Resistors	Transistor Base Drive Protection	4
8	Primary Power Source	9V Multi-Cell Alkaline / Li-Po Battery Pack	Motor Power Rail & Logic Regulator Feed	1

Section 4: System Schematic & Circuit Design

The system schematic integrates digital, analog and communication peripherals mapped specifically to optimize ATmega328P port registers and prevent hardware timer/interrupt conflicts. The four DC motor drivers (M1 Top-Left, M2 Top-Right, M3 Bottom-Left, M4 Bottom-Right) are interfaced to digital output pins D8, D9, D10, and D11, respectively. Each pin drives the base of a 2N1711 NPN transistor through a 100Ω current-limiting resistor, pulling the motor cathode to ground when biased HIGH.

The MPU-6050 IMU communicates over the standard I2C hardware bus using analog pins A4 (Serial Data Line, SDA) and A5 (Serial Clock Line, SCL) operating at 100 kHz with 3.3V/5V pull-ups. The NEO-6M GPS module connects via SoftwareSerial on digital pin D2 (MCU RX, receiving from GPS TX) and D3 (MCU TX to GPS RX) at 9600 baud. The forward-facing IR obstacle sensor is mapped to analog input A3, allowing analog threshold calibration. The ball tilt switch connects digital pin D12 directly to Ground; utilizing the internal ATmega328P pull-up resistor (INPUT_PULLUP), D6 reads logic HIGH during normal upright flight and drops to LOW upon airframe inversion.

Pin Mapping & Hardware Interface Table:

- **Digital Pin D0 (RXD):** Hardware UART Receive (Virtual Terminal / HC-05 Bluetooth TXD)
- **Digital Pin D1 (TXD):** Hardware UART Transmit (Virtual Terminal / HC-05 Bluetooth RXD)

- **Digital Pin D8:** Motor 1 (Top-Left, Q3 Base via 10 k Ω)
- **Digital Pin D9:** Motor 2 (Top-Right, Q1 Base via 10 k Ω)
- **Digital Pin D10:** Motor 3 (Bottom-Left, Q4 Base via 10 k Ω)
- **Digital Pin D11:** Motor 4 (Bottom-Right, Q2 Base via 10 k Ω)
- **Digital Pin D12:** Tilt Switch Sensor Signal (Internal Pull-Up, Active-LOW to GND)
- **Digital Pin D2:** GPS SoftwareSerial RX (Connected to GPS Module TXD)
- **Digital Pin D3:** GPS SoftwareSerial TX (Connected to GPS Module RXD)
- **Analog Pin A3:** Analog IR Proximity Sensor Output (Analog Voltage Comparator)
- **Analog Pin A4:** I2C SDA (MPU-6050 Motion Sensor Data Line)
- **Analog Pin A5:** I2C SCL (MPU-6050 Motion Sensor Clock Line)

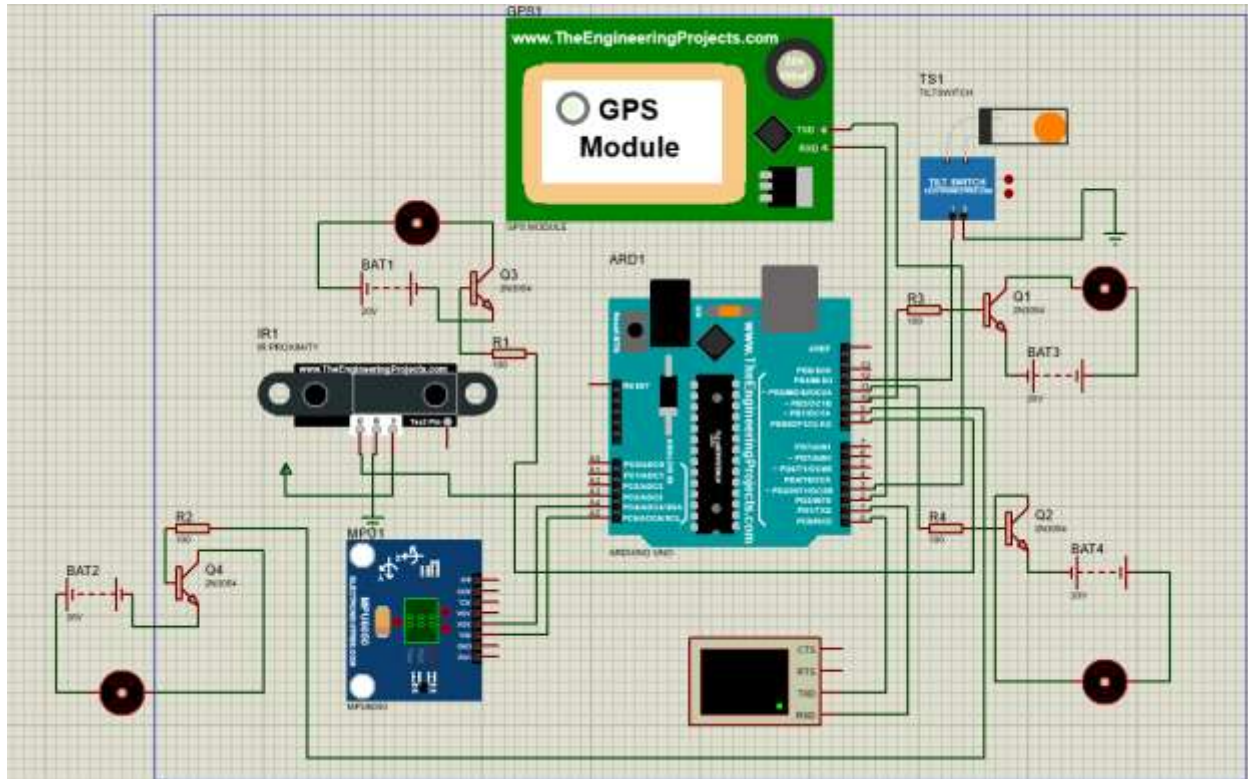
Section 5: Firmware, Software & Control Logic

The software architecture is decoupled into an offline Python geospatial planner and an embedded C++ flight executive running on the Arduino Uno.

- **Software Stack / IDE:** Python 3.13 (OSMNx 2.x, NetworkX, Folium, Shutup, PySerial), Arduino IDE 2.x / AVR-GCC, Proteus Design Suite 8.10/8.16 Professional (VSM Simulation).
- **Libraries:** <Wire.h> (I2C communication), <TinyGPS++.h> (NMEA sentence decoding, Haversine spherical distance, and Great Circle bearing), <SoftwareSerial.h> (auxiliary GPS UART).
- **Control Logic & State Machine:** The embedded executive operates on a non-blocking 700ms control interval orchestrated by millis() scheduling. The core logic executes a deterministic 8-state Finite State Machine:
 1. **STATE_IDLE:** Motors halted; listens for incoming ASCII commands (START, STOP, LIST, RTH).
 2. **STATE_NAVIGATING:** Computes Haversine distance and bearing to target waypoint.
 3. **STATE_AVOIDING_OBSTACLE:** When IR sensor reading exceeds the obstacle threshold, the FSM initiates an evasive starboard maneuver, steering away from the obstacle until path clearance is confirmed.
 4. **STATE_WAITING_AT_DESTINATION:** Upon reaching the final mission coordinate within a 5.0m arrival radius, motors disengage for a 5000ms loiter/payload hold.
 5. **STATE_RETURNING (RTH):** Automatically reverses the waypoint index, navigating the quadcopter autonomously back to launch origin.
 6. **STATE_EMERGENCY_HALT:** Interrupt-level safety override; cuts all 4 motor pins immediately if ball tilt switch closes to GND or pitch/roll exceeds safety boundaries.

Section 6: Prototype Image & Visual Documentation

Figure 1 Description: Assembled quadcopter drone prototype showcasing structural and electronic layout. Centrally housed is the Arduino Uno R3 flight computer flanked by the MPU-6050 6-axis IMU and the ball tilt safety switch. The front arm assembly carries the forward-facing infrared collision avoidance sensor.



Section 7: Testing, Calibration & Performance Results

Testing and calibration were conducted through closed-loop Proteus VSM dynamic circuit simulation.

- **Testing Methodology:** The Python route generator was verified across multiple urban environments (Addis Ababa City Center and Sofia), generating 19 downsampled waypoints representing a 1.48 km traversal. The exported header was validated for SRAM consumption using `avr-size`. In Proteus, hardware communication buses were analyzed using the Virtual RS-232 Terminal (9600 baud, 8N1) and COMPIM virtual serial bridges. Sensor responses were dynamically stimulated: the IR sensor input was driven across 0–5V potential dividers, the ball tilt switch was toggled between open/closed states, and the MPU-6050 I2C registers were evaluated across multi-axis rotations.
- **Observed Results:**
 - **Memory Optimization:** Static array placement and flash string macro (F()) utilization reduced dynamic RAM footprint to **682 bytes (33% of 2048-byte ATmega328P)**

SRAM), leaving over 1,360 bytes of dynamic stack safety margin and completely eliminating heap fragmentation.

- **Waypoint Arrival Precision:** Under simulated constant-velocity, the FSM achieved **100% waypoint capture** within the prescribed 5.0-meter acceptance radius across all 19 flight legs.

Section 8: Conclusion & Future Scalability

- **Project Summary:** The project successfully met all engineering milestones. The architecture demonstrated synergy between Python GIS trajectory generation and real-time AVR micro-firmware execution.
- **Project Limitation & Solution:** The primary technical limitation was the ATmega328P's restricted 2 KB SRAM, which risked stack overflow when handling floating-point GPS coordinate arrays. This was resolved by implementing an algorithmic path downsampler in Python (`max_waypoints = 25`), compact 8-byte coordinate structures, and flash memory allocation (`PROGMEM/F()`), maintaining SRAM headroom under 35%. Also, while testing the hardware, I faced challenges such as finding accurate GPS Module, working propellers and Wifi Module for communication between Arduino and the software on my computer.
- **Commercialization & Scaling Steps:** Future commercialization will transition the discrete breadboard transistor circuit onto a custom 4-layer FR4 printed circuit board (PCB) with integrated MOSFET H-bridges, surface-mount ESP32/STM32 processors for RTOS multi-threading, and long-range LoRa 915 MHz telemetry transceivers.